SR UNIVERSITY

SchoolofComputerScienceandArtificialIntelligence

LabAssignment# 19.2

Program	**:B.Tech (CSE)**
Course Title	**:AIAssistedCoding**
CourseCode	**: 23CS002PC304**
Semester	**II**
Academic Session	**: 2025-2026**
Name of Student	**: P. Vamshi**
Enrollment No.	**:2403A51L24**
Batch No	**: 51**
Date	**:31/03/26**

Task1:

Prompt: Convertthispythonfilehandlingcodeintojava.Useproperjavafilestreamsandexception handling.

PythonCode:

```
file = open("example.txt", "w")
file.write("Hello, this is a sample text file.\n")
file.write("This file is used for demonstrating file handling operations in Python.\n")
file.close()
file = open("example.txt", "r")
content = file.read()
print("Contents of the file:")
print(content)
file.close()
file = open("example.txt", "a")
file.write("This line is added in append mode.\n")
file.close()
file = open("example.txt", "r")
updated_content = file.read()
print("Updated contents of the file:")
print(updated_content)
file.close()
file = open("example.txt", "w")
file.write("This file has been overwritten with new content.\n")
file.close()
file = open("example.txt", "r")
overwritten_content = file.read()
print("Overwritten contents of the file:")
print(overwritten_content)
file.close()
import os
os.remove("example.txt")
```

JavaCode:

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class Assign_19_2 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        String filePath = "example.txt";
        // Write to the file
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
            writer.write(str: "Hello, this is a sample text file.\n");
            writer.write(str: "This file is used for demonstrating file handling operations in Java.\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
        // Read from the file
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String line;
            System.out.println(x: "Contents of the file:");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        // Append to the file
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath, append: true))) {
            writer.write(str: "This line is added in append mode.\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
        // Read the updated contents of the file
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String line;
            System.out.println(x: "Updated contents of the file:");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        // Overwrite the file
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
            writer.write(str: "This file has been overwritten with new content.\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
        // Read the overwritten contents of the file
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String line;
            System.out.println(x: "Overwritten contents of the file:");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        // Delete the file
        File file = new File(filePath);
        if (file.delete()) {
            System.out.println(x: "File deleted successfully.");
        } else {
            System.out.println(x: "Failed to delete the file.");
        }
    }
}
```

PythoncodeOutput:

```
Contents of the file:
Hello, this is a sample text file.
This file is used for demonstrating file handling operations in Python.

Updated contents of the file:
Hello, this is a sample text file.
This file is used for demonstrating file handling operations in Python.
This line is added in append mode.

Overwritten contents of the file:
This file has been overwritten with new content.
```

C++Code Output:

```
Contents of the file:
Hello, this is a sample text file.
This file is used for demonstrating file handling operations in Java.
Updated contents of the file:
Hello, this is a sample text file.
This file is used for demonstrating file handling operations in Java.
This line is added in append mode.
Overwritten contents of the file:
This file has been overwritten with new content.
File deleted successfully.
```

Explanation:

The Python program performs file handling operations such as writing, reading, appending, and deleting a file. It was successfully converted into a Java program using classes like BufferedWriter and BufferedReaderforfileoperations.Exceptionhandlingis implementedusingtry-catchblockstoensuresafe execution. The Java program follows the same logic as the Python version and produces identical output.

Task2:

Prompt:ConvertthisC++sortingprogramintoPython.Maintainsamelogicandoutputandaddsimple comments.

C++ Code:

```cpp
#include <iostream>
using namespace std;
// Function to perform selection sort
void selectionSort(int arr[], int n) {
    int i, j, min_idx;
    // One by one move boundary of unsorted subarray
    for (i = 0; i < n-1; i++) {
        // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
            if (arr[j] < arr[min_idx])
                min_idx = j;
        // Swap the found minimum element with the first element
        swap(arr[min_idx], arr[i]);
    }
}
// Function to print an array
void printArray(int arr[], int size) {
    int i;
    for (i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}
// Driver code
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    selectionSort(arr, n);
    cout << "Sorted array: \n";
    printArray(arr, n);
    return 0;
}
```

PythonCode:

```python
def selection_sort(arr):
    n = len(arr)
    # One by one move boundary of unsorted subarray
    for i in range(n):
        # Find the minimum element in unsorted array
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        # Swap the found minimum element with the first element
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
# Function to print an array
def print_array(arr):
    for i in arr:
        print(i, end=' ')
    print()
# Driver code
if __name__ == "__main__":
    arr = [64, 25, 12, 22, 11]
    selection_sort(arr)
    print("Sorted array: ")
    print_array(arr)
```

C++CodeOutput:

```
Sorted array:
11 12 22 25 64
```

PythonCodeOutput:

```
Sorted array:
11 12 22 25 64
```

Explanation:
TheC++programimplementingSelectionSortissuccessfullyconvertedintoPython.Thelogicremainsthe same, where the smallest element is selected from the unsorted portion and swapped with the current position. Python simplifies the syntax using direct swapping and dynamic arrays (lists). The program correctly sorts the array and prints the output in ascending order. Thus, both implementations produce the same result with equivalent functionality.

Task3:

Prompt: ConvertthisSQLquerywithGROUPBYandaggregatefunctionsintoequivalentPandascode using groupby().

SQL Code:

```sql
CREATE TABLE employees (
    id INTEGER,
    name TEXT,
    department TEXT,
    salary REAL
);

INSERT INTO employees VALUES
(1, 'A', 'HR', 30000),
(2, 'B', 'IT', 50000),
(3, 'C', 'HR', 35000),
(4, 'D', 'IT', 60000),
(5, 'E', 'Sales', 40000);

SELECT department, COUNT(*) AS employee_count, AVG(salary) AS average_salary
FROM employees
GROUP BY department;
```

PythonCode:

```python
import pandas as pd
# Create a DataFrame to represent the employees table
data = {
    'id': [1, 2, 3, 4, 5],
    'name': ['A', 'B', 'C', 'D', 'E'],
    'department': ['HR', 'IT', 'HR', 'IT', 'Sales'],
    'salary': [30000, 50000, 35000, 60000, 40000]
}
employees_df = pd.DataFrame(data)
# Group by department and calculate employee count and average salary
result = employees_df.groupby('department').agg(employee_count=('id', 'count'), average_salary=('salary', 'mean')).reset_index()
print(result)
```

SQL Output:

```
HR|2|32500.0
IT|2|55000.0
Sales|1|40000.0
```

PythonCode Output:

```
  department  employee_count  average_salary
0         HR               2         32500.0
1         IT               2         55000.0
2      Sales               1         40000.0
```

Explanation:
The given SQL query is converted into Pandas using a DataFrame to represent the employees table. The groupby('department') function is used to group data similar to SQL's GROUP BY clause. Aggregate functionslikecountand meanareappliedusingagg()tocalculateemployeecountandaveragesalary.The result is reset into a proper table format using reset_index(). The Pandas output matches the SQL query result, ensuring correct conversion.

Task4:

Prompt: ConvertthisPythonprogramintoanequivalentC++program.Maintainthesamefunctionality and output.

PythonCode:

```
import pandas as pd
# Sample data
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 40, 45],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix']
}
# Create a DataFrame
df = pd.DataFrame(data)
# Display the original DataFrame
print("Original DataFrame:")
print(df)
# Data processing: Filter out rows where Age is greater than 30
filtered_df = df[df['Age'] > 30]
print("\nFiltered DataFrame (Age > 30):")
print(filtered_df)
```

C++code:

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;
// Structure to hold data
struct Person {
    string name;
    int age;
    string city;
};
int main() {
    // Sample data
    vector<Person> people = {
        {"Alice", 25, "New York"},
        {"Bob", 30, "Los Angeles"},
        {"Charlie", 35, "Chicago"},
        {"David", 40, "Houston"},
        {"Eve", 45, "Phoenix"}
    };
    // Display the original data
    cout << "Original Data:" << endl;
    for (const auto& person : people) {
        cout << person.name << ", " << person.age << ", " << person.city << endl;
    }
    // Data processing: Filter out rows where Age is greater than 30
    cout << "\nFiltered Data (Age > 30):" << endl;
    for (const auto& person : people) {
        if (person.age > 30) {
            cout << person.name << ", " << person.age << ", " << person.city << endl;
        }
    }
    return 0;
}
```

PythonCodeOutput:

```
Original DataFrame:
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago
3    David   40      Houston
4      Eve   45      Phoenix

Filtered DataFrame (Age > 30):
      Name  Age     City
2  Charlie   35  Chicago
3    David   40  Houston
4      Eve   45  Phoenix
```

```
Original Data:
Alice, 25, New York
Bob, 30, Los Angeles
Charlie, 35, Chicago
David, 40, Houston
Eve, 45, Phoenix

Filtered Data (Age > 30):
Charlie, 35, Chicago
David, 40, Houston
Eve, 45, Phoenix
```

Explanation:

Python codeuses a DataFrame to store tabular data and filter rows, while the C++ version uses a struct Personandavectortoholdthesamedata.Printingallrowsisdonewithaloopinsteadofprint(df), and filtering (Age > 30) is handled with an if statement inside the loop. Essentially, C++ requires manual structuring and iteration, whereas Python handles it automatically with pandas.

Task5:

Prompt: ConvertthisPythoneven/oddfunctionintoaC++program.

PythonCode:

```python
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Test cases
test_cases = [10, 15, 0]
for test in test_cases:
    result = check_even_odd(test)
    print(f"{test} is {result}.")
```

C++Code:

```
    print(f"{test} is {result}.")
#include <iostream>
#include <vector>
#include <string>
std::string check_even_odd(int num) {
    if (num % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}
int main() {
    std::vector<int> test_cases = {10, 15, 0};
    for (int test : test_cases) {
        std::string result = check_even_odd(test);
        std::cout << test << " is " << result << "." << std::endl;
    }
    return 0;
}
```

PythonCode Output:

```
10 is Even.
15 is Odd.
0 is Even.
```

C++Code Output:

```
10 is Even.
15 is Odd.
0 is Even.
```

Explanation:

- Thefunctioncheck_even_oddtakesanintegerandreturns"Even"or"Odd"basedon%2.
- Thetest_casesvectorstoresnumberstocheck, likethe Python list.
- Arange-basedforloop iteratesovereachnumber intest_cases.
- std::coutprintstheresultinthesameformatasPython’s print.
- TheprogrammimicsPython behaviourbutusesC++syntaxandtypes(std::string,std::vector).